Security and Provenance

Security and Provenance I

Research integrity requires more than reproducibility; it requires verifiable authorship. In an era of generative AI, automated scraping, and synthetic media, the ability to prove that a document was produced by a specific pipeline at a specific time is a critical defense against fabrication and misattribution. The W3C PROV data model (Moreau and Missier 2013) establishes a formal vocabulary for expressing provenance records—entities, activities, and agents connected by derivation, generation, and attribution relations. Digital watermarking, pioneered by Cox et al. (Cox et al. 1997) for multimedia integrity verification, provides the foundational signal-processing theory for embedding imperceptible provenance markers within artifacts. `template/` implements a domain-specific provenance layer that embeds these relations directly into the PDF artifact itself, using four complementary steganographic and cryptographic mechanisms.

Threat Model

Threat Model I

The steganography subsystem defends against three classes of threats:

Threat Model I

1. **Unauthorized redistribution**: A manuscript is scraped and republished without attribution. The steganographic watermark survives the redistribution and can be used to prove original authorship.
2. **Content tampering**: Figures or text are modified after publication. The SHA-256 hash embedded in the PDF metadata detects any modification to the file contents.
3. **Provenance forgery**: An attacker claims to have produced a document they did not author. The build timestamp, commit hash, and pipeline metadata embedded in the watermark provide a verifiable chain of custody.

Steganographic Layers

Steganographic Layers I

The system applies four complementary layers of provenance information:

Layer 1: PDF Metadata Injection I

The `inject_pdf_metadata` function writes structured metadata into both the PDF Info dictionary and an XMP (Extensible Metadata Platform) packet:

Layer 1: PDF Metadata Injection I

- /Creator: Research Template Steganography Module
- /Producer: Module path (infrastructure.steganography)
- /CreationDate / /ModDate: UTC timestamp in ISO 8601 format
- /Author: From config.yaml
- /Title: From config.yaml
- /Subject: document identifier, plus /DocumentID
- Dedicated /Hash_<ALGO> fields for each computed digest (e.g. /Hash_SHA256)
- Custom fields: GitCommit, GitCommitAvailable, /SteganographyTimestamp
- An embedded stego_manifest.json attachment carrying the document id, digests, title, and git commit

Layer 2: Cryptographic Hashing I

Before watermarking, SHA-256 (and SHA-512 when configured) digests of the original PDF are computed and stored in:

Layer 2: Cryptographic Hashing I

- A JSON hash manifest sidecar (`<pdf>.hashes.json`, e.g. `output/pdf/<name>_combined.hashes.json`)
- Dedicated PDF metadata fields (`/Hash_SHA256`, `/Hash_SHA512`)
- The embedded `stego_manifest.json` attachment

This enables post-hoc verification: anyone with the manifest can verify that the PDF has not been modified since rendering.

Layer 3: Alpha-Channel Overlay, Footer, and Invisible Text I

A semi-transparent overlay is merged onto every page of the hardened PDF:

Layer 3: Alpha-Channel Overlay, Footer, and Invisible Text

- **Watermark overlay**: in the default `text` mode, a configurable string (default `CONFIDENTIAL`) is repeated diagonally across the page; in `qr` mode a tiled QR overlay replaces it. The overlay is rendered at low opacity — the default is 8% (`overlay_opacity: 0.08`), configurable from 0.02 (subtle) to 0.30 (strong) — to be invisible during normal viewing but detectable through image analysis.
- **Provenance footer**: every page additionally receives a footer carrying the document identifier, page number / total pages, a digest prefix, title, authors, and source filename/size.
- **Invisible text layer**: the first page embeds a hidden text string with the machine-readable payload:

```
STEG_ID:<document-id>|TITLE:<title>|HASHES:<hash-prefix>
```

Layer 3: Alpha-Channel Overlay, Footer, and Invisible Text I

The build timestamp and Git commit hash (short SHA) are recorded in the PDF metadata and the hash manifest rather than in the visible watermark text itself. Together these fields allow a verifier to reconstruct—from the hardened PDF alone—which version of the code, at which moment in time, produced the document.

Layer 4: QR and Barcode Injection I

When barcodes are enabled, a bottom-of-page barcode strip (QR code + Code128 label) is merged onto **every** page, encoding the document identifier and page number (`ID:<doc-id-prefix>|P:<page>`). In `overlay_mode: "qr"`, a tiled QR overlay covers the full page in place of the text watermark. Both are configurable via the `steganography:` block in `manuscript/config.yaml` (or the repo-level `secure_config.yaml` defaults).

The `secure_run.sh` Orchestrator

The `secure_run.sh` Orchestrator I

The steganographic pipeline is orchestrated by `secure_run.sh`, a Bash script that wraps the standard `run.sh` pipeline with post-processing steganography:

The `secure_run.sh` Orchestrator I

1. Execute the standard YAML-declared pipeline (17 stages; default full 10) pipeline for the target project.
2. The `secure_run.sh` script invokes `SteganographyProcessor`.
3. Apply metadata injection, hashing, text overlay, and QR code injection.
4. Save the secured PDF alongside the original.
5. Generate a steganography report in JSON format.

The `secure_run.sh` Orchestrator I

The orchestrator processes either a single specified project or all discovered projects sequentially.

Tamper Detection

Tamper Detection I

Verification is performed by comparing the stored SHA-256 hash against a freshly computed hash of the distributed PDF. Any modification—even a single bit flip—produces a hash mismatch. The alpha-channel overlay provides a secondary, visual verification channel that does not require access to the original hash.

Limitations

Limitations I

- Alpha-channel overlays are stripped by some PDF optimization tools (e.g., `qpdf --optimize`).
- QR codes are visible and may be removed by a determined attacker.
- The system does not provide non-repudiation in the cryptographic sense—it does not use digital signatures with private keys. Future versions may integrate GPG or X.509 signing.
- The provenance model is pipeline-centric rather than fully PROV-compliant. The path from `template/`'s current metadata-based provenance to full W3C PROV-compliant traces involves four steps: (1) *entity identification*—assigning stable identifiers (URIs or SWHIDs) to each pipeline input (manuscript files, data, config); (2) *activity logging*—recording each pipeline stage as a PROV Activity with start/end timestamps (already encoded in the watermark overlay); (3) *agent attribution*—binding each Activity to the pipeline version and Git commit hash (already encoded in the overlay and PDF metadata); (4) *PROV-O serialization*—emitting the provenance graph as OWL-RDF (PROV-O) or text (PROV-N) alongside the PDF. Steps (2) and (3) are already implemented; steps (1) and (4) are the primary remaining gaps. A future `infrastructure.provenance` module would close both gaps automatically.

Relationship to Software Supply Chain Integrity

Relationship to Software Supply Chain Integrity I

The steganographic provenance layer operates at the *document level*—it certifies the integrity of a specific PDF artifact. A complementary concern is *build-level* provenance: certifying that the pipeline itself was executed with verified source code and dependencies. Frameworks such as in-toto (Torres-Arias et al. 2019) and SLSA (Supply-chain Levels for Software Artifacts) address this concern by defining attestation chains from source commit through build steps to final artifact. The NTIA's minimum elements for a Software Bill of Materials (SBOM) (National Telecommunications and Information Administration 2021) further standardize the enumeration of software components and dependencies—essential for establishing the provenance lineage of build environments. SLSA defines four graduated levels of build integrity:

Relationship to Software Supply Chain Integrity I

SLSA Level	Requirement	`template/` Status
1	Provenance document exists	✓ SHA-256 manifest + steganographic metadata
2	Version-controlled build scripts	✓ All scripts in git
3	Isolated build environment	~ Docker support exists but not enforced in CI
4	Hermetic, reproducible builds	N – future work

Relationship to Software Supply Chain Integrity I

Future versions of `template/` may generate SLSA-compatible provenance attestations alongside the steganographic watermarks, creating a two-layer provenance model: in-toto attests that the build pipeline was executed with the claimed source code, while the steganographic layer attests that the PDF was produced by that pipeline at a specific time.

Relationship to FAIR and Formal Provenance Standards

Relationship to FAIR and Formal Provenance Standards I

The steganographic layer supports the FAIR for Research Software (FAIR4RS) principles (Barker et al. 2022) at the artifact level. PDFs carry embedded metadata (Findability) in standardized XMP format (Interoperability). The SHA-256 hash manifest enables persistent integrity verification (a prerequisite for Reusability). The Documentation Duality standard ensures that the software producing the artifact is inspectable and well-documented (satisfying FAIRsoft (Garijo et al. 2024) metadata and documentation indicators). Full PROV-compliant provenance traces—capturing the derivation chain from source data through analysis scripts to rendered PDF—are a natural extension and a primary target for future development.

Relationship to FAIR and Formal Provenance Standards I

Software Heritage (Di Cosmo et al. 2020) complements this picture at the source-code level: by archiving the `template/` repository and assigning a reproducible SWHID (Software Hash Identifier) to each commit, Software Heritage makes the pipeline itself—not just its output—a citable, persistent digital artifact. A published SWHID alongside the PDF DOI creates a complete, two-artifact citation record: the paper's content is versioned via DOI; the code that generated it is versioned via SWHID. This combination satisfies the Katz et al. (Katz et al. 2021) software citation principles' requirement that software used in research be independently citable and permanently accessible.